Resources
Typeclasseopedia: http://wiki.haskell.org/Typeclassopedia#Abstract
Applicative programming with effects (First definition of Applicative): https://www.staff.city.ac.uk/~ross/papers/Applicative.html

Symbols
a → b
a × b
x ↦ v

Functor
Applicative
Monad
Functor: https://downloads.haskell.org/ghc/latest/docs/libraries/base-4.17.0.0/Prelude.html
Monad: https://downloads.haskell.org/ghc/latest/docs/libraries/base-4.17.0.0/Prelude.html
Applicative: https://downloads.haskell.org/ghc/latest/docs/libraries/base-4.17.0.0/Control-Applicative.html

```
-- Ref: https://downloads.haskell.org/ghc/latest/docs/libraries/base-4.17.0.0/src/GHC.Base.html
class Functor f where
    fmap        :: (a -> b) -> f a -> f b

class Functor f => Applicative f where
    {-# MINIMAL pure, ((<*>) | liftA2) #-}
    pure :: a -> f a
    (<*>) :: f (a -> b) -> f a -> f b

class Applicative m => Monad m where
    {-# MINIMAL pure, ((<*>) | liftA2) #-}
    (>>=)       :: forall a b. m a -> (a -> m b) -> m b
    (>>)        :: forall a b. m a -> m b -> m b
    return      :: a -> m a
    return      = pure
```

Basic idea
A **data structure** represents a **container** of **values**.
A data structure is a **functor** if the usual **map operation** can be applied on the inner values.
Applicative/monad are **generalizations** of the functor.
i.e. applicative/monad operations are generalization of the **map operation**.
  functor=simple maps --> monad=generic map.

In particular, a data structure is used to represent an **action** + **action result**.
A monad is a data structure representing an action which can be **chained**
i.e. can describe a **sequence of actions**
i.e.
  container type m= **action type**
  inner value type a= **action result type**
  monadic function (k :: a -> m b) = function computing **next action structure**
  monad bind (x >>= k) = **process** action structure x and **chain** to next structure

e.g. IO a = a **runtime** read/write action with result type a.
e.g. Maybe a = a **success** (Just) or **fail** (Nothing) check action

Just 1 = success with result 1
Nothing = fail with no result
e.g. Either a = like Maybe, a success (Right) or error (Left) check action with a custom error value.
Right 1 = success with result 1
Left (-1) = error with error value -1
e.g. (-> r) i.e. function = a mapping action
(*2) = a map Int → Int i.e. result=integers
e.g. [] i.e. list = all possible **outcomes** of a computation
[1,2,3,4] = a computation where outcome=1,2,3 or 4.

e.g. JavaScript **Promise** + async/await is a form of monad.
Promise = an asynchronous action type
await operation = (>>=)

```
-- Haskell
do
 putStrLn "Enter name"
 name <- getLine
 putStrLn $ "Hello" ++ name

-- as JS Promise
async function do() {
 await putStrLn("Enter name");
 const name = await getLine();
 await putStrLn(`Hello ${name}`);
}
```

Functor <$> vs Applicative <*> vs Monad >>=
- Functor (<$>): usual **map**
i.e. a **unary mapping** f:a→b of the result value

```
f <$> mx                          apply f::a -> b on mx::m a

-- e.g.
(*2) <$> Just 1        -- Just 2
(*2) <$> Nothing       -- Nothing
(*2) <$> [1,2]         -- [2,4]
(*2) <$> (+1)          -- 2(x+1)
(*2) <$> (+1) $ 10     -- 22
```

- Applicative (<*>): same as functor, but with **multiple variables**
i.e. a **multivariate mapping** f:a0 × a1 × a2 × ... →b of result values
i.e. a type of action where **multiple results** can be **combined into a single result**

```
pure f <*> mx <*>  my <*> mz       apply f::a -> b -> c -> d on
mx::m a, my::m b, mz:: m c

-- e.g. binary
f = (*)
pure f <*> Just 2 <*> Just 3       -- Just 6
pure f <*> Nothing <*> Just 3      -- Nothing
pure f <*> [1,2] <*> [3,4]         -- [1*3,1*4,2*3,2*4] = [3,4,6,8]
```

```
pure f <*> (+1) <*> (+2)           -- (x+1)(x+2)
pure f <*> (+1) <*> (+2) $ 10      -- 132

-- e.g. ternary
f x y z = x * y * z
pure f <*> Just 2 <*> Just 3 <*> Just 4     -- Just 24
pure f <*> [1,2] <*> [3,4] <*> [1,10]       --
 [3,30,4,40,6,60,8,80]
pure f <*> (+1) <*> (+2) <*> (+3)           -- (x+1)(x+2)(x+3)
```

- Monad (>>=): most generic map
 i.e. instead of just mapping the result value, **compute next action structure** directly

```
mx >>=  k                          apply k::a -> m b on mx:: m a

-- e.g.
Just 1 >>= (\x -> Just (2*x))       -- Just 2
Nothing >>= (\x -> Just (2*x))      -- Nothing
[1,2] >>= (\x -> [1..2*x])          -- [1,2] ++ [1,2,3,4] =
[1,2,1,2,3,4]
(+1) >>= (\t -> ((2*t)+))           -- 2(x+1) + x
(+1) >>= (\t -> ((2*t)+)) $ 10      -- 32
```

- pure x/return x: a **trivial** action result
 i.e. an action with **constant** result

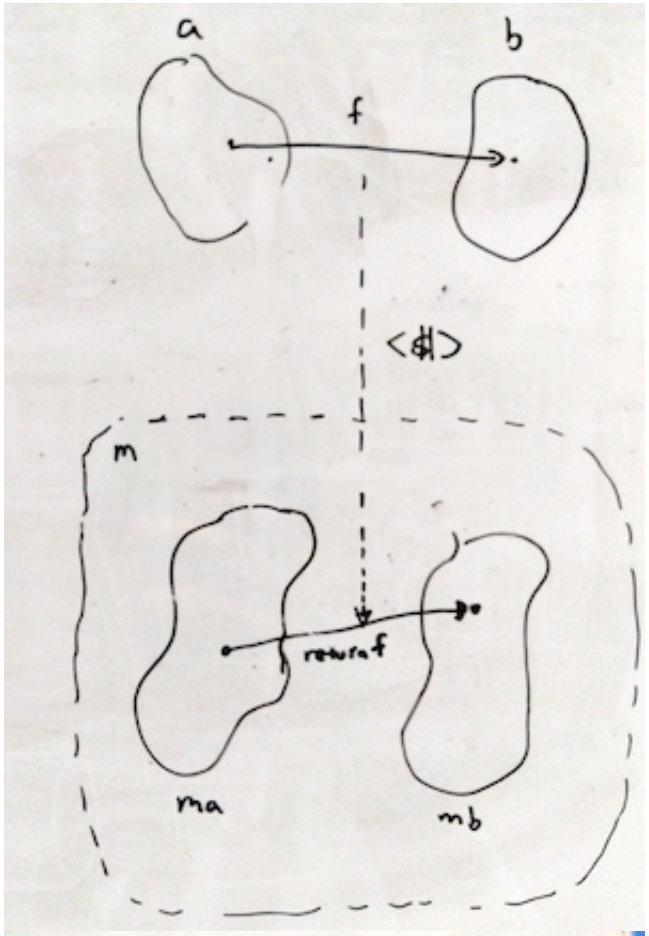
```
-- e.g.
return 1 :: Just Int             -- Just 1 i.e. a constant success
return 1 :: Either Int b         -- Right 1 i.e. a constant success
return 1 :: [Int]                -- [1] i.e. singleton
return 1 :: Int -> Int           -- f x = 1 i.e. constant function
```

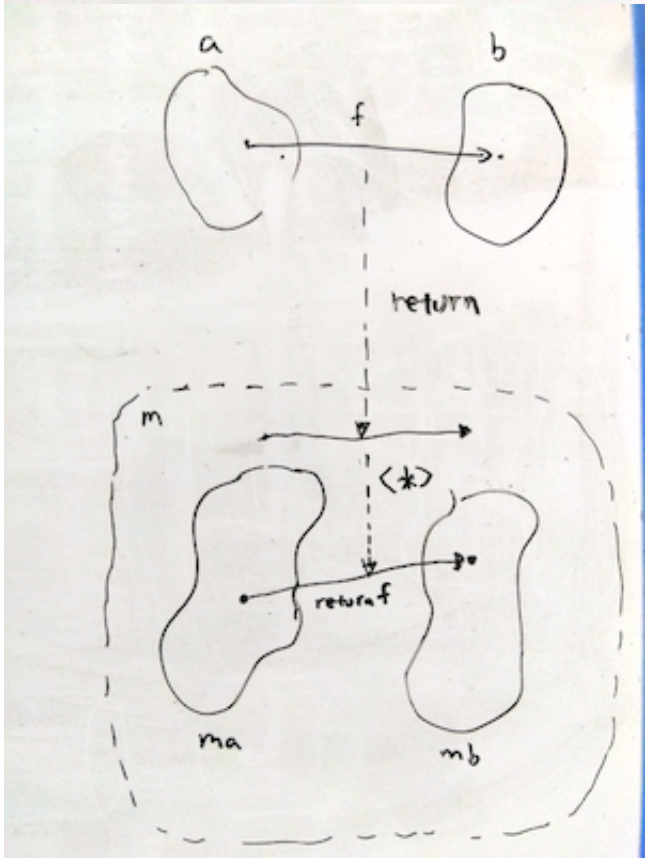
	Elementary	Generalization
constant value	x :: a	return x :: m a pure x
function application	f x f::a->b, x::a	mf <*> mx mf::m (a -> b), mx::m a -- simple case with f:a->b f <$> mx pure f <*> mx -- same f::a -> b, mx::m a

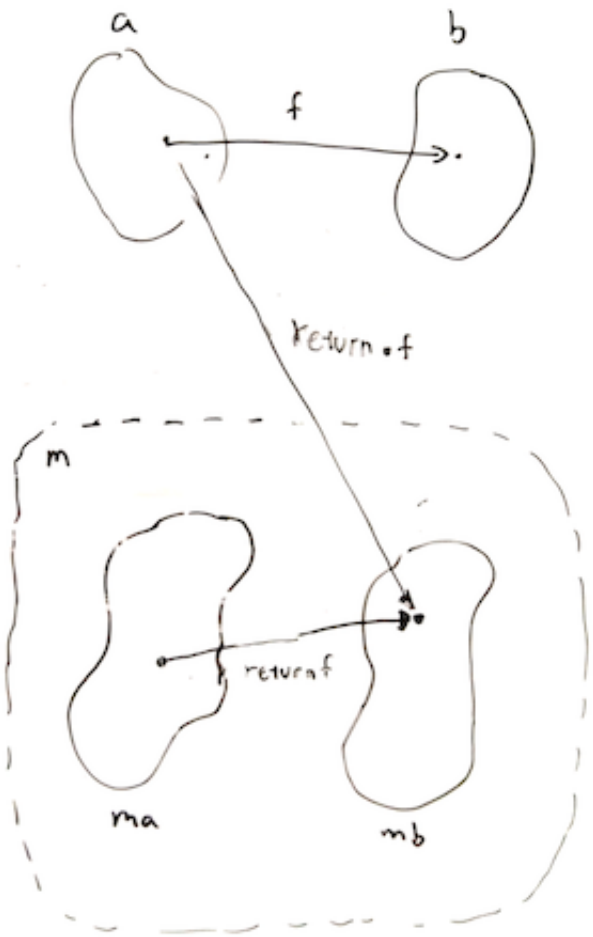
multivariate application	f x y z f::a->b->c, x::a, y::b, z::c	mf <*> mx <*> my <*> mz mf::m (a->b->c), mx::m a, my::m b, mz::m c -- simple case with f::a -> b -> c f <$> mx <*> my <*> mz pure f <*> mx <*> my <*> mz -- same f::a -> b -> c, mx::m a, my::m b, mz::m c
nested application	h (g (f x)) h::c->d, g::b->c, f::a->b, x::a	mh <*> (mg <*> (mf <*> mx)) mh::m (c->d), mg::m (b->c), mf::m (a->b), mx::m a -- same as monad mx >>= (\x -> mf >>= (\f -> mg >>= (\g -> mh >>= (\h -> return h (g (f x))))))
composition Special cases of multivariate with (.) g f i.e. F(g,f) where F=(.)	((.) g f) x == g (f x) g::b->c, f::a->b, x::a	(pure (.) <*> mg <*> mf) <*> mx == mg <*> (mf <*> mx) mg::m (b->c), mf::m (a->b), mx::m a
nested computation	h (g (f x)) h::c->d, g::b->c, f::a->b, x::a	mx >>= kf >>= kg >>= kh kh::c -> m d, kb:: b -> m c, kf::a -> m b, mx::a
sequence of expressions	seq x y	mx >> my mx *> my mx <* analogous to seq (seq a b) a i.e. (mx >> my) >> mx

Functor vs applicative vs monad

```
x :: m a
f :: a -> b
                f <$> x            ==         (return f) <*> x
  ==         x >>= (return . f)
```


a
b
f
<$>
m
returnf
ma
mb


a
b
f
return
m
<*>
returnf
ma
mb



Laws

```
Functor Laws
             fmap id   ==   id
         fmap (f . g)   ==   fmap f . fmap g



Applicative Laws
       pure id <*>
m   ==   m                                    Identity
   mf <*> (mg <*> m)   ==  (pure (.) <*> mf <*> mg) <*>
m    Associativity
                                                                  i.e.
analogue of f (g x) = (f . g) x = ((.) f g) x
    pure f <*> pure x  ==  pure (f
x)                        Homomorphism
         mf <*> pure y ==  pure ($ y) <*>
mf                  Interchange
                                                                  i.e.
analogue of ($ y) f

Monad Laws
      return x >>= kf  ==  kf x                                left
identity
         m >>= return  ==  m                                     right
identity
    (m >>= kf) >>= kg  ==  m >>= (\x -> (kf x >>=
```

```
kg))      associativity
In particular, if m = kf, then
      return x >>= kf  ==  m
         m >>= return  ==  m
```

Relationships

```
Relationships
-- Applicative
              f <$> m  ==  pure f <*> m
           i.e. (<$>)  ==  (<*>) .
pure                            map = lift f + apply

-- Monad
f <$> m :: Monad m => (a -> b) -> m a -> m b
              f <$> m  ==  fmap f m
                       ==  m >>= return . f
mf <*> m :: Monad m => m (a -> b) -> m a -> m b
             mf <*> m  ==  mf >>= (\f -> f <$> m)
 IMPORTANT: order matters. i.e. mf is executed first
                       ==  mf >>= (<$>m)
```

==========

Examples

Example: Identity a
Ref: https://downloads.haskell.org/ghc/latest/docs/libraries/base-4.17.0.0/src/Data.Functor.Identity.html#line-128

```
-- | @since 4.8.0
instance Functor Identity where
    fmap     = coerce
-- | @since 4.8.0
instance Applicative Identity where
    pure     = Identity
    (<*>)    = coerce
    liftA2   = coerce
-- | @since 4.8.0
instance Monad Identity where
    m >>= k  = k (runIdentity m)
```

Example: Maybe a
Ref: https://downloads.haskell.org/ghc/latest/docs/libraries/base-4.17.0.0/src/GHC.Base.html

```
-- | @since 2.01
```

```
instance  Functor Maybe  where
    fmap _ Nothing       = Nothing
    fmap f (Just a)      = Just (f a)

-- | @since 2.01
instance Applicative Maybe where
    pure = Just
    Just f  <*> m        = fmap f m
    Nothing <*> _m       = Nothing
    liftA2 f (Just x) (Just y) = Just (f x y)
    liftA2 _ _ _ = Nothing
    Just _m1 *> m2       = m2
    Nothing  *> _m2      = Nothing

-- | @since 2.01
instance  Monad Maybe  where
    (Just x) >>= k       = k x
    Nothing  >>= _       = Nothing
    (>>) = (*>)
```

Example: List [a]
Ref: https://downloads.haskell.org/ghc/latest/docs/libraries/base-4.17.0.0/src/GHC.Base.html

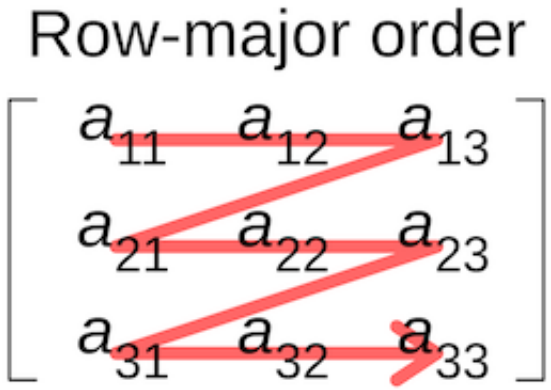
```
-- | @since 2.01
instance Functor [] where
    {-# INLINE fmap #-}
    fmap = map

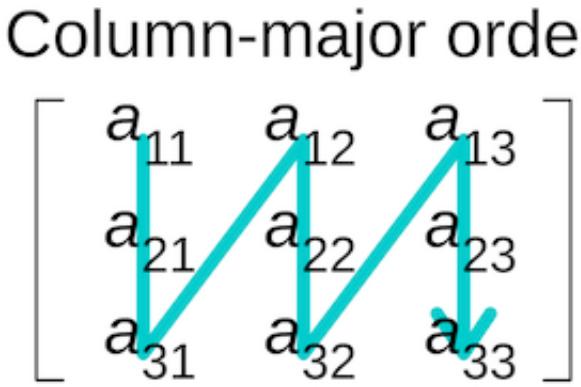
-- | @since 2.01
instance Applicative [] where
    {-# INLINE pure #-}
    pure x    = [x]
    {-# INLINE (<*>) #-}
    fs <*> xs = [f x | f <- fs, x <- xs]
    {-# INLINE liftA2 #-}
    liftA2 f xs ys = [f x y | x <- xs, y <- ys]
    {-# INLINE (*>) #-}
    xs *> ys  = [y | _ <- xs, y <- ys]

-- See Note: [List comprehensions and inlining]
-- | @since 2.01
instance Monad []  where
    {-# INLINE (>>=) #-}
    xs >>= f             = [y | x <- xs, y <- f x]
    {-# INLINE (>>) #-}
    (>>) = (*>)
```

Basic idea

action = all possible **outcomes** of a **computation**.
i.e. all possible **combination** of input values
e.g. **cartesian product** of input values
Results are collapsed in **row major** order.






```
e.g.
-- cartesian product
id <$> [True,False]                                        --
[True,False]
(,) <$> [True,False] <*> ['a'..'c']                        --
 [(True,'a'),(True,'b'),(True,'c'),(False,'a'),(False,'b'),
(False,'c')]
(,,) <$> [True,False] <*> ['a'..'c'] <*> [1..3]            --
 [(True,'a',1),(True,'a',2),(True,'a',3),

 (True,'b',1),(True,'b',2),(True,'b',3),

(True,'c',1),(True,'c',2),(True,'c',3),

(False,'a',1),(False,'a',2),(False,'a',3),

(False,'b',1),(False,'b',2),(False,'b',3),

(False,'c',1),(False,'c',2),(False,'c',3)]
(,) <$> [] <*> ['a'..'c']                                  -- []
(,,) <$> [] <*> ['a'..'c'] <*> [1..5]                      -- []

-- ragged matrix
[1..4] >>= \x-> [1..x]                  -- [1,1,2,1,2,3,1,2,3,4]
[1..4] >>= \x-> zip [x,x..] [1..x]      -- [(1,1),(2,1),(2,2),
(3,1),(3,2),(3,3),(4,1),(4,2),(4,3),(4,4)]
```

f <$> x = apply f:a → b on each value of x
i.e. 1 x n matrix
mf <*> x = apply f_i:a → b on each value of x + collapse to row major
i.e. m x n matrix
x >>= k = apply k:a → [b] on each value of x + collapse to row major

```
f <$> x
|f|  |x0,x1,...,xn|  = |f x0|  1 x n matrix
                      |f x1|
                      | ...  |
                      |f xn|

                    = row major form |f x0, f x1, f x2,..., f xn|
                      length = n

mf <*> x
|f0|  |x0,x1,...,xn| = |f0 x0, f0 x1, f0 x2, ... |  square m x n
matrix
|f1|                 |f1 x0, f1 x1, f1 x2, ... |
|f2|                 |f2 x0, f2 x1, f2 x2, ... |
|... |                 |        ...               |
|fm|                 |fm x0, fm x1, fm x2, ... |

                    = row major form |f0 x0, f0 x1, f0 x0,..., f1
x1, f1 x2,..., fm xn|
                      length = mn

mx >>= mf
mf(|x0,x1,...,xn|)   = | f_0 x0 |
                      | f_1 x1 |
                      | ...      |
                      | f_n xn |

                    = ragged matrix
                      | f_0,0 x0, f_0,1 x0,...,   |  upto f_0,m0
                      | f_1,0 x1, f_1,1 x1, ... ... |  upto f_1,m1
                      | f_2,0 x2, f_2,1 x2, ... ... |  upto f_2,m2
                      | ...                         |
                      | f_n,0 xn, f_n,1 xn, ... ... |  upto f_n,mn

                    = row major form |f_0,0 x0, f_0,1 x0,..., f_n,0
xn, f_n,1 xn,...|
                      length = m0 + m1 + ... + mn
```

e.g.

```
-- Cartesian product
(0,) <$> [1,2,3]
[(0,)] <*> [1,2,3]                      -- [(0,1),(0,2),(0,3)
(,) <$> [1,2,3] <*> [4,5,6]             -- [(1,4),(1,5),(1,6),
(2,4),(2,5),(2,6),(3,4),(3,5),(3,6)]
```

```
[(1,),(2,),(3,)] <*> [4,5,6]            -- same
(,,) <$> [1,2,3] <*> [4,5,6] <*> [7,8]  -- [(1,4,7),(1,4,8),
(1,5,7),(1,5,8),(1,6,7),... ,(3,6,7),(3,6,8)]
[(1,4,),(1,5,)(1,6),(2,4,)...] <*> [7,8]  -- same

-- short circuits if empty
[] >>= return [1..10]     -- []
[] >> [1..10]             -- same
[1] >>= return [1..10]    -- [1..10]
[1] >> [1..10]            -- same
[1,2] >>= return [1..10]  -- [1..10,1..10]
[1,2] >> [1..10]


-- special case: flatten
[[1,2,3],[4,5]] >>= id       -- [1,2,3,4,5]
join [[1,2,3],[4,5]]         -- same
msum [[1,2,3],[4,5]]
```

Example: zipList [a] (Applicative only)

```
newtype ZipList a = ZipList { getZipList :: [a] }
                  deriving ( Show     -- ^ @since 4.7.0.0
                           , Eq       -- ^ @since 4.7.0.0
                           , Ord      -- ^ @since 4.7.0.0
                           , Read     -- ^ @since 4.7.0.0
                           , Functor  -- ^ @since 2.01
                           , Foldable -- ^ @since 4.9.0.0
                           , Generic  -- ^ @since 4.7.0.0
                           , Generic1 -- ^ @since 4.7.0.0
                           )
-- @since 2.01
instance Applicative ZipList where
    pure x = ZipList (repeat x)
    liftA2 f (ZipList xs) (ZipList ys) = ZipList (zipWith f xs ys)

-- | @since 4.11.0.0
instance Alternative ZipList where
   empty = ZipList []
   ZipList xs0 <|> ZipList ys0 = ZipList $ go xs0 ys0
     where
       go (x:xs) (_:ys) = x : go xs ys
       go    []     ys  = ys
       go    xs      _  = xs
```

Basic idea
A zipList is a simpler definition of a list as an **applicative**.
A zipList is NOT a monad
action = **term-wise** computation on a list
i.e. the same as the **zip** function
i.e. operations preserve the **same length**
albeit like zip, result = length of the shorter list.

```
e.g.
ZipList [(*1),(*2)] <*> ZipList [10,100]            --  =ZipList
[10,200] i.e. termwise application
[(*1),(*2)] <*> [10,100]                            --
= [10,100,20,200] i.e. matrix product application

-- truncates to shorter like zip
ZipList [(*1),(*2),(*3)] <*> ZipList [10,100]       - =
ZipList [10,200]
ZipList [(*1)] <*> ZipList [10,100]                 -- = ZipList
[10]
```

Example: ((->) r a)
i.e. { u: u: r → a }
Ref: https://downloads.haskell.org/ghc/latest/docs/libraries/base-4.17.0.0/src/GHC.Base.html

```
-- instances for Prelude types
-- | @since 2.01
instance Functor ((->) r) where
    fmap = (.)

-- | @since 2.01
instance Applicative ((->) r) where
    pure = const
    (<*>) f g x = f x (g x)
    liftA2 q f g x = q (f x) (g x)

-- | @since 2.01
instance Monad ((->) r) where
    f >>= k = \ r -> k (f r) r
```

Basic idea
action = Function on (->) r a) defined as function **composition** on the range
return v = constant function x ↦ v
<$> = usual **composition**
i.e. f <$> g = f(g(x))
i.e. <$> = (.)
<*> = **nary composition**
i.e. pure u <*> f <*> g <*> h = u(f(x),g(x),h(x))
>>= = most generic composition
i.e. compose g with a unique f_a(g(x)) for each a=g(x)

>>= is actually <*> just flipped

```
-- proof
(>>=) f k x = k (f x) x        -- By definition
(<*>) k f x = k x (f x)        -- By definition

(>>=) f (flip k) x
```

```
   == (flip k) (f x) x
   == k x (f x)
   == (<*>) k f x
-- i.e.
f >>= (flip k) == k <*> f
```

e.g.

```
-- nested composition
--
-- via
--   u <*> (v <*> (w <*> x))
-- e.g. linear transform f(x)
pure (*3)                   -- f(y)=3y
pure (*3) <*> (+5)          -- f(x)=3(x+5)
(*3) <$> (+5)               -- same
(*3) . (+5)
(+5) >>= return . (*3)
pure (+1) <*> (pure (*3) <*> (+5))                    --
 f(x)=3(x+5)+1
pure (*2) <*> (pure (+1) <*> (pure (*3) <*> (+5)))    --
f(x)=2(3(x+5)+1)
(*2) . (+1) . (*3) . (+5)                             -- same
pure (*2) <*> (pure (+1) <*> (pure (*3) <*> (+5))) $ 1 --
38=2(3(1+5)+1)


-- multivariate composition
-- i.e. particular, allows multiple compositions
-- via
--   u <*> v <*> w <*> x        (left associative)
--
-- e.g. binary
pure (*)                        -- f(y,z)=yz
pure (*) <*> (+1)               -- f(x+1,z)=(x+1)z
pure (*) <*> (+1) <*> (+2)      -- f(x+1,x+2)=(x+1)(x+2)
(pure (*) <*> (+1)) <*> (+2)    -- same. Left associative
(*) <$> (+1) <*> (+2)           -- same
((*) . (+1)) <*> (+2)           -- same
do x <- (+1); y <- (+2); return (x*y)    -- same
pure (*) <*> (+1) <*> (+2) $ 1   -- 6=(1+1)(1+2)

-- e.g. 4 variables
g=(\t u v w  -> t * u * v * w)
pure g <*> (+1)                                 --
g(x+1,u,v,w)=(x+1)uvw
pure g <*> (+1) <*> (+2)                        --
 g(x+1,x+2,v,w)=(x+1)(x+2)vw
pure g <*> (+1) <*> (+2) <*> (+3)               --
 g(x+1,x+2,x+3,w)=(x+1)(x+2)(x+3)w
pure g <*> (+1) <*> (+2) <*> (+3) <*> (+4)      --
 g(x+1,x+2,x+3,x+4)=(x+1)(x+2)(x+3)(x+4)
pure g <*> (+1) <*> (+2) <*> (+3) <*> (+4) $ 1   -- 120=(1+1)(1+2)
```

```
(1+3)(1+4)


---- >>=
(+10) >>= (*)           -- (x+10)x
(*) <*> (+10)           -- same

-- do
f = do          -- x(x+3)
 u <- (+3)
 (*u)
f = (+3) >>= (\u -> (*u))
f = do          -- (x+1)(x+3)+x
 u <- (+3)      -- u=x+3
 v <- (+1)      -- v=x=1
 (u*v+)
f = (+3) >>= (\u -> ((+1) >>= (\v -> (u*v+))))
```

Example: IO
Ref: https://downloads.haskell.org/ghc/latest/docs/libraries/base-4.17.0.0/src/GHC.Base.html

```
-- | @since 2.01
instance  Functor IO where
   fmap f x = x >>= (pure . f)

-- | @since 2.01
instance Applicative IO where
    {-# INLINE pure #-}
    {-# INLINE (*>) #-}
    {-# INLINE liftA2 #-}
    pure  = returnIO
    (*>)  = thenIO
    (<*>) = ap
    liftA2 = liftM2

-- | @since 2.01
instance  Monad IO  where
    {-# INLINE (>>)   #-}
    {-# INLINE (>>=)  #-}
    (>>)      = (*>)
    (>>=)     = bindIO


returnIO :: a -> IO a
returnIO x = IO (\ s -> (# s, x #))

bindIO :: IO a -> (a -> IO b) -> IO b
bindIO (IO m) k = IO (\ s -> case m s of (# new_s, a #) -> unIO (k
a) new_s)
```

```
thenIO :: IO a -> IO b -> IO b
thenIO (IO m) k = IO (\ s -> case m s of (# new_s, _ #) -> unIO k
new_s)

unIO :: IO a -> (State# RealWorld -> (# State# RealWorld, a #))
unIO (IO a) = a

...

ap                 :: (Monad m) => m (a -> b) -> m a -> m b
ap m1 m2           = do { x1 <- m1; x2 <- m2; return (x1 x2) }
```

==========

OLD

i.e.
For any u: r → a, transform u to a v: r → b

return 0	=	x ↦ 0	i.e. a constant function
f <$> u	=	x ↦ f(u(x))	f: a → b
F <*> u	=	x ↦ F(x)(u(x))	F(x): a → b for x ∈ r
	=	u >>= flip F	
u >>= F	=	x ↦ F(u(x), x)	F: a × r → b